CMPE3209 Computer Organization — Proje Teslim Tutanağı

Kullanım talimatı

Bu tutanak projenin tek resmi teslim belgesidir. Her kilometre taşında (M1, M2 ve final; tümü pazartesi) ilgili adım doldurulur ve aynı dosya güncellenerek yeniden teslim edilir. Ayrıntılı görev tanımları proje yönergesindedir; bu tutanak yalnızca ne istendiğini ve nereye yazılacağını gösterir.

- Tüm sayılar birimiyle yazılır (byte, cycle, ns, GHz). Hex değerler 0x önekiyle yazılır.
- **Varsayım** olan her değer "(varsayım)" etiketiyle işaretlenir.
- Tablolara sığmayan çıktılar (assembly listesi, objdump, CSV izi, ekran görüntüleri) Git deposuna konur. Tutanakta dosya adıyla atıf yapılır.
- Boş bırakılan zorunlu alan, o adımın puanını sıfırlar.

Öğrenci bilgileri

Alan	Değer
Ad Soyad	Caner Sivri
Öğrenci numarası (SID)	5001230018
E-posta	canersivri@ogr.iu.edu.tr
Seçilen görev (G1–G5)	G2
Seçilen ISA ve sürümü (ör. RV32IM)	RISC-V RV32IM
Derleyici ve sürümü	riscv64-unknown-elf-gcc (-march=rv32im -mabi=ilp32)
Simülatör / araçlar	Ripes v2.3.1
Git deposu adresi	https://github.com/CanerSivri/CMPE3209-G2

Adım 0 — Kişiye özel parametreler

Bu adım M1 ile birlikte teslim edilir. Hatalı bir parametre, ona dayanan tüm adımları geçersiz kılar; öğretim üyesi bu tabloyu M1'de kontrol eder.

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
d	SID'in son hanesi	5001230018 % 10	8
D2	SID'in son iki hanesi	5001230018 % 100	18
S	SID'in hanelerinin toplamı	5+0+0+1+2+3+ 0+0+1+8	20
N	24 + (D2 mod 25)	24 + (18 % 25)	42
Cache	d'ye göre: boyut / line / mapping / line sayısı / set sayısı	256 bytes / 8 bytes / 2-way / 32 lines / 16 sets	256/8/ 2-way/32 /16
Write policy	d çift → write-back, d tek → write-through	d is even	write-back
k	(S mod 7) + 3	(20 % 7) + 3	9
Interrupt türü	S çift → Timer, S tek → I/O	20 is even	Timer
f	1.0 + 0.1 × d GHz (varsayım)	1.0 + 0.1 x 8	1.8 GHz (assum ption)
T1 / T2	T1 = 1 cycle; T2 = 20 + 5 × (d mod 4) cycle (varsayım)	T1=1 cycle T2 = 20 + 5 x (8%4) = 20 cycles	T1= 1 cycle , T2 = 20 cycles (assum ption)
Nc	2 + (S mod 7) (varsayım)	2 + (20 % 7) = 8	8 cores (assum ption)
Görev parametresi	Seçilen göreve göre (T, P, K, yön veya R)	G2: 50 + (5001230018 % 10) = 58	P = 58

Adres alanları (ISA'nın adres genişliği s + w bit; 64-bit ISA'da düşük 32 bit kullanılır):

Alan	Bit sayısı	Hesap
Word (w)	3 bits	$\log_2$(8bytes)

Alan	Bit sayısı	Hesap
Line (r) veya set (d)	4 bits	$\log_2$(16 sets)
Tag (s − r veya s − d)	25 bits	32-(3+4) bits
Toplam	32 bits	25 + 4 + 3 bits

Adım 1 — Görev seçimi ve C programı (M1: 2 Kasım, Pazartesi)

1.1 Görev seçimi (yalnızca birini işaretleyin):

- ☐ G1 Biyomedikal sinyal alarmı
- × G2 Sınav notu dağılımı
- ☐ G3 Mesaj şifreleme ve bütünlük
- ☐ G4 Sıralama ve medyan
- ☐ G5 Run-length sıkıştırma

1.2 Çözüm tasarımı (3–5 cümle: hangi algoritma, hangi fonksiyonlar, hangi veri yerleşimi ve neden):

> *The program generates the 42 grades with the LCG using my SID and stores them in the global array grades[N], computing the sum, the pass count (P = 58, calculated by guidelines) and the five bucket counts in the same first loop. After the integer mean is computed, a second loop calls mark_below(note, avg) for every element; the function returns the grade with bit 8 set (OR 0x100) if it is below the mean, and main stores the returned value and increments below_count only when it differs from the original. All results (grades, bucket[5], mean, below_count, pass_count) are global variables so they have fixed addresses in the data section and can be checked directly in the simulator's memory view. Counting in the first pass avoids a third loop, and setting bit 8 marks a grade without losing its original value in the low 8 bits.*

1.3 C kaynak kodu. Satır numaralı kod aşağıdaki bloka yapıştırılır (25–60 satır):

```
 1  /* Caner Sivri, 5001230018, G2 - Exam Grade Distribution*/
 2  #define N 42
 3
 4  const int SID = 1230018;
 5  int grades[N];
 6  int bucket[5];
 7  int mean = 0;
 8  int below_count = 0;
 9  int pass_count = 0;
10  int mark_below(int note, int avg);
11
12  int main(void){
13      unsigned int x = (unsigned int)SID;
14      int pass_limit = 50 + (SID % 10);
15      int sum = 0;
16
17      for(int i=0;i<N;i++){
18          x = 1664525U * x + 1013904223U;
19          int v = (int)(((x >> 16) % 101U) + 0U);
20          grades[i] = v;
21          sum += v;
```

```
22
23          if(v >= pass_limit){
24              pass_count++;
25          }
26          if(v < 50){
27              bucket[0]++;
28          }
29          else if(v < 65){
30              bucket[1]++;
31          }
32          else if(v < 75){
33              bucket[2]++;
34          }
35          else if(v < 85){
36              bucket [3]++;
37          }
38          else{
39              bucket[4]++;
40          }
41      }
42
43      mean = sum / N;
44
45      for(int i=0;i<N;i++){
46          int marked = mark_below(grades[i],mean);
47          if(marked != grades[i]){
48              grades[i] = marked;
49              below_count++;
50          }
51      }
52      return 0;
53  }
54
55  int mark_below(int note, int avg){
56      if (note < avg){
57          return note | 0x100;
58      }
59      return note;
60  }
```

1.4 Zorunlu yapı eşlemesi. Her madde için satır numarası ve tek cümlelik açıklama:

#	Zorunlu yapı	Satır no	Açıklama
Y1	N elemanlı global int dizisi, en az iki geçiş	5,20,48	A 42-element global grades array is defined and traversed using two separate loops for marking and populating.
Y2	Koşullu dallanma	23,26,29,32,35,38,47,56	Pass limit, bucket ranges, below-mean checks, and the threshold comparison are

#	Zorunlu yapı	Satır no	Açıklama
			implemented
Y3	Parametreli, değer döndüren fonksiyon çağrısı	10,46,55	Function is declared and called with mark_below(grades[i],mean) and returns grade.
Y4	Toplama/çıkarma, çarpma veya shift, bit işlemi	18,19,57	Multiplication, addition and shift operations are used in LCG and bitwise or used in mark_below function to store flag.
Y5	Dizi elemanına store	20,48	Generated grades and marked grades are stored back in global array.
Y6	const int SID ve hesapta kullanımı	4,13,14	SID constant is used in LCG calculations and passing threshold limit.
Y7	Sonucun tutulduğu global değişken(ler)	6,7,8,9	Distribution,mean,below mean and pass count are global variables .

1.5 Program çıktıları. Simülatörde çalıştırıldıktan sonra bellekten okunan değerler:

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
bucket[0..4]	0x0001145C – 0x0001146C	0x17,0x05,0x07,0x02,0x05	screenshots/output1.png

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
mean	0x000113A8	0x30	screenshots/output2.png
below_count	0x000113AC	0x16	screenshots/output2.png
pass_count	0x000113B0	0x13	screenshots/output2.png

1.6 İzlenecek kernel (12–20 dinamik instruction; en az bir load, store, ALU, koşullu branch ve call/return içermeli):

Alan	Değer
C satır aralığı	60 -> 46-48
Başlangıç ve bitiş adresi (hex)	0x000103A0 – 0x00010314
Dinamik instruction sayısı	17
Hangi döngü iterasyonu (ör. i = 3)	second loop,i=0(45th line)

Öğretim üyesi (M1): ☐ Onaylandı ☐ Düzeltme istendi ☐ Reddedildi — Tarih: _____ Not: _____

Adım 2 — Assembly ve machine code (M2: 9 Kasım, Pazartesi)

2a. C → assembly eşlemesi (`-O0`; listenin tamamı depoda `asm_O0.s`):

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)

Bölüm	Başlangıç adresi (hex)	Boyut (byte)	İçerik
`.text`			
`.data` / `.bss`			

Von Neumann ilkesi bu adreslerde nasıl görünüyor? (1–2 cümle)

2b. `-O0` ve `-O1` karşılaştırması (kernel için):

Ölçü	-O0	-O1	Farkın nedeni
Kernel'deki instruction sayısı			
Load sayısı			
Store sayısı			
Kullanılan register sayısı			

2c. Elle machine code (en az 10 instruction; x86-64 için 8). Kullanılan tüm formatlar kapsanmalı.

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
1							
2							
3							
4							
5							
6							
7							
8							
9							
10							

Zorunlu instruction'ların tablodaki sıra numarası:

- ☐ SID'i yükleyen instruction(lar): # ____
- ☐ Koşullu branch: # ____
- ☐ Load: # ____
- ☐ Store: # ____
- ☐ Fonksiyon çağrısı: # ____

SID'in yüklenmesi. Sabitin neden tek instruction'a sığmadığı, nasıl bölündüğü ve gerekiyorsa +1 düzeltmenin sign extension ile açıklaması:

Branch offset hesabı. Branch'in adresi, hedef adres ve offset'in adım adım hesabı:

Adım 3 — CPU, interrupt ve bus (final paketiyle: 16 Kasım, Pazartesi)

Model varsayımları (okudum ve uyguladım):

- ☐ Pipeline yok, her an tek instruction yürütülür.
- ☐ PC, IR, MAR ve MBR Fig. 3.2'deki gibi kavramsal olarak vardır.
- ☐ Her bellek erişimi bir bus işlemidir; cache bu adımda yok sayılır.
- ☐ Data bus genişliği = ____ bit.

Modelin gerçek donanımdan ayrıldığı yerler (3–5 cümle):

3a. Instruction cycle izleme tablosu. Her dinamik instruction bir satırdır. Bir instruction birden çok bellek erişimi yapıyorsa MAR, MBR ve bus hücrelerine sırayla yazılır. Kategori: PM = processor–memory, PIO = processor–I/O, DP = data processing, C = control.

#	PC (önce → sonra)	Assembly	IR (hex)	Fig. 3.6 durumları	MAR	MBR	ALU işlemi	Değişen register	Bus (adres / data / control)	Kategori
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										
11										
12										
13										
14										
15										
16										
17										

#	PC (önce → sonra)	Assembly	IR (hex)	Fig. 3.6 durumları	MAR	MBR	ALU işlemi	Değişen register	Bus (adres / data / control)	Kategori
18										
19										
20										

Kernel toplamları	Sayı
Instruction fetch	
Operand fetch (of)	
Operand store (os)	
PC'nin sıra dışı değiştiği instruction	

3b. Fig. 3.4 makinesine aktarım. Kernel'in tek bir dizi elemanını güncelleyen parçası, opcode 1, 2, 3, 5 ve 7 ile yeniden yazılır ve Fig. 3.5 formatında izlenir:

Adım	Fetch / execute	PC	IR (hex)	AC	Değişen bellek hücresi
1					
2					
3					
4					
5					
6					
7					
8					

Ölçü	Seçilen ISA	Fig. 3.4 makinesi
Aynı iş için instruction cycle sayısı		

Tek accumulator'ın getirdiği sınırlama (PDP-11 ADD B,A ile karşılaştırarak):

IAS register'ları ve seçilen ISA:

IAS register'ı	IAS'taki görevi	Seçilen ISA'daki karşılığı	Programcıya görünür mü?
MBR			
MAR			
IR			
IBR			
PC			
AC			
MQ			

3c. Interrupt senaryosu (k = ____, tür = ____):

Soru	Cevap
Interrupt hangi instruction'ın execute'u sırasında geliyor? (adres, assembly)	
Ne zaman tanınır ve bu instruction neden yarıda kesilmez? (Fig. 3.9)	
Kaydedilen PC değeri (hex)	
Kaydedilen diğer durum bilgisi	
Seçilen ISA'daki gerçek mekanizma (register adları, vektör)	
Kaynak (ISA referansı, bölüm, sayfa)	
Handler dönüşünde ilk çalışan instruction'ın adresi	

Fonksiyon çağrısı (Y3) ve interrupt karşılaştırması:

Ölçüt	Fonksiyon çağrısı	Interrupt
Akışı kim başlatır?		
Dönüş adresini kim kaydeder, nereye?		
User program bundan haberdar mı?		

İkinci (daha yüksek öncelikli) interrupt. Aynı olay dizisini iki yaklaşımla zaman çizelgesi olarak doldurun (Fig. 3.13 ve 3.14):

Zaman	Sequential (disable)	Nested (priority)
t_0: 1. interrupt gelir		
t_1: 2. interrupt gelir		
t_2		
t_3		

Zaman	Sequential (disable)	Nested (priority)
t_4: user program'a dönüş		

3d. Bus ve interconnection.

Bus işlemi türü (kernel, cache yok)	Sayı	Aktif control line'lar
Instruction fetch (read)		
Data read		
Data write		
Toplam		

Hesap	Değer	Adımlar
Address bus genişliği ve maksimum adreslenebilir bellek		
Data bus 16 bit olsaydı kernel'in bus cycle sayısı		
Mevcut / 16-bit oranı		

PCIe paket verimi (N × 4 = ____ byte'lık tek Memory Write TLP; Fig. 3.25):

Alan (octet)	12-byte header, ECRC yok	16-byte header, ECRC var
STP framing (baş)		
Sequence number		
Header		
Data		
ECRC		
LCRC		
STP framing (son)		
Toplam		
Verim = data / toplam		
128b/130b ile verim		

Sonucun bir çekirdekten PCIe cihazına yolu (Fig. 3.17 ve 3.21): her adımda bağlantı, protokol katmanı ve aktarım birimi.

Adım 4 — Cache simülasyonu (final paketiyle: 16 Kasım, Pazartesi)

4a. Data adres izi. Tüm load ve store'lar depoda trace.csv dosyasındadır. Biçim: sira,RW,adres_hex,boyut,c_satiri,stack_mi.

Ölçü	Değer
Kullanılan derleme (-O0 / -O1)	
Toplam data referansı	
Load / store	
Stack erişimi sayısı	
Ana diziye erişim sayısı	

4b. İlk 5 farklı adresin alanları:

Adres (hex)	Adres (binary)	Tag	Line / Set	Word

4c. Elle simülasyon (ilk 30 referans):

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
12								
13								
14								
15								
16								
17								
18								
19								
20								
21								
22								
23								
24								
25								
26								
27								
28								
29								
30								

Cache'in 30. referanstan sonraki anı. Line sayınız 8'den fazlaysa satır ekleyin.

Line	Set	Valid	Tag (hex)	Dirty (write-back)	LRU sırası (associative)
0					
1					
2					
3					
4					
5					

Line	Set	Valid	Tag (hex)	Dirty (write-back)	LRU sırası (associative)
6					
7					

4d. Tam simülasyon sonuçları:

Ölçü	Elle (ilk 30)	Araç (ilk 30)	Araç (tüm iz)
Referans	30	30	
Hit			
Miss			
Hit ratio H			
Ana belleğe yazma sayısı			

Kullanılan araç ve ayarları (kendi simülatörünüz varsa dosya adı):

- ☐ Elle ve araçla yapılan simülasyon ilk 30 referansta birebir örtüşüyor.

4e. Tasarım uzayı (cache boyu sabit):

Mapping (line boyu sabit = ____ B)	Hit	Miss	H
Direct			
2-way set-associative			
4-way set-associative			
Fully associative (LRU)			

Line boyu (mapping sabit = ____)	Hit	Miss	H
8 B			
16 B			
32 B			
64 B			

Grafik dosyası ve eğilimlerin locality ile açıklaması (thrashing varsa çatışan adresler):

4f. Ortalama erişim süresi: $T_s = H \times T1 + (1 - H) \times (T1 + T2)$

Durum	H	T_s (cycle)	Speedup (cache'siz duruma göre)

Durum	H	T_s (cycle)	Speedup (cache'siz duruma göre)
Cache yok	0	T2 =	1
Kişiye özel cache			

4g. Kod değişikliği:

	Önce	Sonra
Değiştirilen C satır(lar)ı		
Y7 çıktısı (aynı olmalı)		
H		
T_s (cycle)		

Değişikliğin gerekçesi; H artmadıysa nedeni:

Adım 5 — Performans ve architecture/organization (final paketiyle: 16 Kasım, Pazartesi)

5a-1. Dinamik instruction dağılımı ve CPI (CPI_i değerleri yönergedeki model varsayımıdır):

Sınıf	CPI_i (varsayım)	I_i (-O0)	I_i (-O1)
ALU	1		
Çarpma	3		
Load	1		
Store	1		
Branch / jump	2		
Ic (toplam)			
Ortalama CPI			

5a-2. Süre ve MIPS rate (f = ____ GHz, τ = ____ ns):

Ölçü	Formül	-O0	-O1
T (bellek etkisi yok)	$Ic \times CPI \times \tau$		
MIPS rate	$f / (CPI \times 10^6)$		
m (instruction başına data referansı)	data ref. / Ic		
T, cache yok (k = T2)	$Ic \times [p + (m \times k)] \times \tau$		
T, cache var (k = T_s)	$Ic \times [p + (m \times k)] \times \tau$		

Karşılaştırma	Değer
Speedup -O0 → -O1 (Eq. 2.1, cache var)	
Speedup cache yok → cache var (-O0)	

MIPS rate'in iki derlemeyi karşılaştırmada neden yanıltıcı olabileceği, kendi sayılarınızla:

5a-3. Amdahl yasası (Nc = ____):

Alan	Değer
Paralelleştirilebilen kısım (C satırları)	
Bağımsızlık gerekçesi (veri bağımlılığı)	
f (yürütme süresindeki pay)	
Speedup = 1 / [(1 − f) + f / Nc]	
Nc → ∞ limiti	

5b. Architecture ve organization (en az 10 özellik):

#	Özellik	Architecture / Organization	Gerekçe (Stallings tanımı)	Değişseydi Y7 değişir miydi? Kanıt
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				

Ch. 1.2'deki dört temel fonksiyon:

Fonksiyon	Kernel'den örnek instruction (adres, assembly)	Açıklama
Data processing		
Data storage		

Fonksiyon	Kernel'den örnek instruction (adres, assembly)	Açıklama
Data movement		
Control		

Final teslim (16 Kasım Pazartesi; sözlü savunma 20 Kasım Cuma)

Teslim kontrol listesi:

- ☐ Bu tutanak, tüm adımları dolu olarak (PDF ve düzenlenebilir sürüm)
- ☐ C kaynak kodu (main.c) ve sonraki 4g sürümü (main_v2.c)
- ☐ asm_O0.s, asm_O1.s ve objdump -d çıktısı
- ☐ trace.csv ve cache simülasyon çıktıları (kendi simülatörünüz varsa kaynak kodu)
- ☐ Ekran görüntüleri: machine code doğrulaması, program çıktıları, cache durumu
- ☐ 4e grafiği
- ☐ Git deposunda M1, M2 ve final etiketleri

Kilometre taşı kaydı (öğretim üyesi doldurur):

Kilometre taşı	Son tarih	Teslim tarihi	Git etiketi	Paraf
M1 Öneri (Adım 0–1)	2 Kasım (Pzt)			
M2 Assembly ve machine code (Adım 2)	9 Kasım (Pzt)			
M3 CPU, interrupt, bus (Adım 3)	16 Kasım (Pzt, final paketiyle)			
M4 Cache ve performans (Adım 4–5)	16 Kasım (Pzt, final paketiyle)			
Final ve sözlü savunma	Teslim 16 Kasım (Pzt); savunma 20 Kasım (Cuma)			

YZ kullanım beyanı. Kullanılmadıysa ilk satıra "Kullanılmadı" yazın. Beyan edilmemiş kullanım akademik dürüstlük ihlalidir.

Araç ve sürümü	Adım	Ne için kullanıldı	Örnek prompt	Çıktıyı nasıl doğruladım
Claude Opus 5.5, Gemini Argon 4	1.2	Çözüm tasarımı bölümünde grammar ve teknik hataları düzeltmek	"Check my solution proposal and fix any mistakes grammarly or technically"	Claude'den alınan çıktıyı yeni bir Claude sekmesinde ve Gemini'de herhangi bir hata var mı şeklinde sorarak doğruladım.
Claude Opus 5.5	0	ISA ve derleyici kurulumu sırasında	"How to download and	Kullanım sırasında herhangi bir hata ile karşılaşmadım.

Araç ve sürümü	Adım	Ne için kullanıldı	Örnek prompt	Çıktıyı nasıl doğruladım
		ayarlamalar yapmak için	setup ISA and compiler”	

Öğrenci beyanı. Bu tutanaktaki C kodunu, elle machine code kodlamasını, instruction cycle izleme tablosunu ve elle cache simülasyonunu kendim yaptım. Başka bir öğrencinin çalışmasını kopyalamadım ve kendi çalışmamı paylaşmadım. YZ kullanımımı eksiksiz beyan ettim. Sözlü savunmada bu çalışmayı açıklayamamamın puanımı etkileyeceğini biliyorum.

Ad Soyad	Öğrenci numarası	Tarih	İmza

Değerlendirme (öğretim üyesi doldurur)

Bileşen	Azami	Puan	Not
A1 Öneri ve parametre hesabı (Adım 0–1)	5		
A2 Assembly ve machine code (Adım 2)	15		
A3 CPU, interrupt, bus (Adım 3)	25		
A4 Cache (Adım 4)	25		
A5 Performans ve architecture/organization (Adım 5)	10		
Tutanak ve rapor kalitesi	5		
Sözlü savunma	15		
Geç teslim kesintisi	–		
Toplam	**100**		

Sözlü savunma kaydı:

Madde	Sorulan	Sonuç (başarılı / kısmen / başarısız)	Puan
1. Canlı encode (-O1 listesinden)			
2. Adres alanları ve hit/miss			
3. Interrupt k+1'de gelseydi			
4. Tasarım kararı savunması			

☐ 1. ve 2. maddelerin ikisi de başarısız: A2–A4 puanları 0.5 ile çarpıldı.

Öğretim üyesi	Tarih	İmza